Oracle C2M & WebLogic Patching & Upgrade Runbook (to 25.4)

Implementation Solution Guide

Version 1.0.0 · 2026-07-06

Contents

Executive Summary — Oracle C2M & WebLogic Patching & Upgrade Runbook (to 25.4) 4

What you get . . . 4

What this delivers . . . 4

Version boundary . . . 4

The deployment model . . . 5

Outcome . . . 5

1. Solution Overview — Oracle C2M & WebLogic Patching & Upgrade Runbook (to 25.4) 5

What this runbook is . . . 5

Why this matters . . . 5

Supportability — why staying current is not optional . . . 5

The certified-version matrix . . . 5

The tooling — OPatch and the OUAF utilities . . . 6

The environment progression (lower first) . . . 6

Patch vs upgrade — same discipline, different scope . . . 6

What this is not . . . 6

The deployment model . . . 7

2. Requirements Guide — Oracle C2M & WebLogic Patching & Upgrade Runbook (to 25.4) 7

2.1 My Oracle Support (MOS) access and the patch/upgrade media . . . 7

2.2 Certified target release and stack . . . 7

2.3 Verified backups (domain config, OUAF product/config, database) . . . 7

2.4 The patch README / upgrade guide read and understood . . . 8

2.5 Change control and a maintenance window . . . 8

2.6 A non-production pilot environment . . . 8

2.7 Administrative access and tooling (OPatch, OUAF utilities, SQL, WLST) . . . 8

2.8 A go/no-go gate and rollback plan per environment . . . 8

Checklist . . . 9

Global Requirements — Oracle Utilities C2M / WebLogic Solutions 9

1. Licensed Oracle Utilities C2M + WebLogic entitlement . . . 9

2. Certified platform stack (OS, Java, Database, WebLogic) . . . 10

3. Environment topology (Dev / Test / Prod) & access . . . 10

4. Administrative access & tooling (WLST, shell, SQL) . . . 10

5. Change control, backup & a non-production pilot . . . 10

6. Collaborating teams & support model . . . 11

3. Upgrade & Patch Runbook — Oracle C2M & WebLogic Patching & Upgrade Runbook (to 25.4) 11

Phase A — Readiness and backup (every environment, before any apply) . . . 11
Phase B — Apply the patch / perform the upgrade . . . 12
Phase C — Post-apply validation (every environment) . . . 13
Phase D — Rollback (if a gate fails) . . . 14
Phase E — Promote up, then production . . . 14
5. Testing Guide — Oracle C2M & WebLogic Patching & Upgrade Runbook (to 25.4) 14
Test 1 — Backup was verified before apply (readiness) . . . 15
Test 2 — WebLogic patch inventory shows the change . . . 15
Test 3 — C2M / OUAF inventory shows the change . . . 15
Test 4 — Managed servers reach RUNNING . . . 15
Test 5 — Database is at the expected level . . . 15
Test 6 — C2M functional smoke test . . . 15
Test 7 — Coverage report is clean and matches baseline intent . . . 15
Test 8 — Rollback path is confirmed available . . . 15
Pass/fail gate . . . 16
6. Troubleshooting Guide — Oracle C2M & WebLogic Patching & Upgrade Runbook (to 25.4) 16
A. OPatch apply fails or reports a conflict . . . 16
B. OUAF patch install fails . . . 16
C. Managed servers do not reach RUNNING after apply . . . 17
D. Backup/pre-check gate will not pass . . . 17
E. Stack is not certified for the target release . . . 17
F. Version/patch drift between nodes . . . 18
G. A change failed validation — rollback . . . 18
H. Diagnostics quick reference . . . 18
I. When to escalate . . . 18
7. Best Practices — Oracle C2M & WebLogic Patching & Upgrade Runbook (to 25.4) 18
8. Deliverables Summary — Oracle C2M & WebLogic Patching & Upgrade Runbook (to 25.4) 19
Documentation . . . 19
Automation scripts (WLST/Jython, shell & SQL — not PowerShell) . . . 20
Named runbook artifacts . . . 20
What the buyer receives . . . 20
Outcome . . . 21
Appendix A — PowerShell Scripts 22

Executive Summary — Oracle C2M & WebLogic Patching & Upgrade Runbook (to 25.4)

Keeping Oracle Utilities Customer to Meter (C2M) supportable means staying current: applying Oracle patches to C2M/OUAF and to Oracle WebLogic Server, and periodically performing version upgrades (for example, toward **C2M 25.4**). Done ad hoc, this is where production outages come from — a patch applied without a backup, an upgrade run straight against production, a change with no rollback. This solution is the disciplined method to do it safely: readiness, backup, apply, validate, rollback, proven in lower environments before production.

This is a complete, do-it-today runbook written for a C2M/WebLogic administrator. It treats every patch and every upgrade as the same governed operation — the repeatable way — so the outcome does not depend on who is on shift.

What you get

• A plain-language Solution Overview of the C2M/OUAF + WebLogic patch and upgrade model:

supportability, the certified-version matrix, the OPatch and OUAF install/upgrade utilities, and the environment progression (lower environments first).

• A Requirements Guide with verify / implement-if-missing for every prerequisite — My Oracle Support

(MOS) patch access, verified backups, a certified target release, change control, and a non-production pilot environment.

• An Upgrade & Patch Runbook of precise, ordered steps: readiness and backup, apply Oracle patches or

perform the version upgrade, post-apply validation, and documented rollback — per environment with explicit go/no-go gates.

• Five automation scripts (WLST/Jython, shell, and SQL) for read-only patch/version discovery,

read-only post-patch validation, a read-only health/coverage report, a dry-run-by-default remediation helper, and a scope/target helper — authentic to C2M and WebLogic administration.

• Testing and Troubleshooting guides with explicit pass/fail gates, plus field-proven Best Practices

for staying current safely.

What this delivers

C2M and WebLogic kept current and secure — patches and upgrades applied with readiness, backup, validation, and a documented rollback — proven in lower environments before production, so you close support gaps without unplanned outages.

Version boundary

Version-specific details (C2M 25.4 / minimum OUAF v2.9, WebLogic 14c) are cited or flagged "verify for your version." Specific patch numbers and MOS notes are pointed to on My Oracle Support, not reproduced here — always read each patch's own README before applying it.

The deployment model

Assess, then Configure, then Pilot, then Validate, then Expand — every patch and upgrade proven in a non-production environment before production, with a backup and a documented rollback at every step.

Outcome

A supportable, current C2M/WebLogic estate: patches and version upgrades applied on a disciplined cadence with go/no-go gates and a rollback you have actually tested — replacing risky, one-off patching with a repeatable runbook.

1. Solution Overview — Oracle C2M & WebLogic Patching & Upgrade Runbook (to 25.4)

What this runbook is

Oracle Utilities Customer to Meter (C2M) is a layered product: the Oracle Utilities Application Framework (OUAF) and the C2M application on top, deployed on Oracle WebLogic Server, backed by an Oracle Database, running on a certified OS and JDK. Keeping it supportable means keeping every layer current — applying Oracle **patches** (single fixes, rollups, service packs, and WebLogic patch-set updates) and periodically performing **version upgrades** (for example, toward **C2M 25.4**). This runbook is not a one-time script; it is the disciplined, repeatable method to apply patches and perform upgrades across environments without taking production down: readiness, backup, apply, validate, rollback.

Why this matters

Patching and upgrading is where most self-inflicted C2M outages come from. The failure modes are predictable: a patch applied to production with no verified backup and therefore no way back; an upgrade run first (or only) against production, so the first time anyone sees the new behavior is in front of users; a patch applied to an uncertified WebLogic/JDK/DB combination that loses supportability; a "quick" fix with no rollback that leaves the domain half-patched. The fix is not a better one-off — it is a governed operation applied the same way every time: prove it in a lower environment, back up before you touch anything, apply with the vendor's own utilities, validate against a gate, and keep a rollback you have actually tested. This section is the why; the Implementation Guide (titled *Upgrade & Patch Runbook*) is the how.

Supportability — why staying current is not optional

Oracle support and security are tied to release and patch level. Running an out-of-support C2M/OUAF release or an unpatched WebLogic means security exposure and, when you open a service request, a "patch to a supported level first" response before Oracle will engage. Staying current on a disciplined cadence keeps you inside the support window and closes known vulnerabilities before they are exploited — which is the entire point of doing this deliberately rather than reactively.

The certified-version matrix

Every C2M release is certified against specific OS, JDK, Oracle Database, and WebLogic versions. C2M 25.4 (minimum OUAF v2.9, WebLogic 14c — **verify for your version** against the release's certification matrix on My Oracle Support) only supports certified combinations. Before any upgrade you confirm the target C2M release and align OS/JDK/DB/WebLogic to a certified combination; before any patch you confirm the patch is for your release. Patching or upgrading into an uncertified stack is how installs fail and supportability is lost.

The tooling — OPatch and the OUAF utilities

Two families of tooling do the work, and this runbook uses them, not a GUI-only workflow:

- **WebLogic / Fusion Middleware — OPatch.** `opatch lsinventory` reports what is applied to a

WebLogic Oracle Home; `opatch apply` applies a patch; `opatch rollback` removes it. OPatch is shipped in the Oracle Home (`$ORACLE_HOME/OPatch`); WebLogic 14c uses the OPatch version shipped with that release (**verify for your version**). Always read the patch README first.

- **C2M / OUAF — the framework utilities.** `configureEnv.sh` sets the environment configuration;

`initialSetup.sh` applies configuration to the product/domain; `installSF.sh` and `installSFgroup.sh` install single/group software fixes; `ouafDatabasePatch.sh` applies database patches; `spl.sh` starts and stops the environment. OUAF installs the database component first, then the application-server component, then runs an error-checker — and shuts WebLogic environments down automatically during install (non-WebLogic environments are stopped manually).

The included automation is authored in **WLST (Jython), shell, and SQL** — the same interfaces an administrator uses — never PowerShell.

The environment progression (lower first)

Nothing is applied to production first. The progression is fixed: apply and validate in the lowest environment (Dev), promote to the next (Test/QA/UAT), and only after each lower environment passes its gate does the change reach Production — inside a change window, with a verified backup and a documented rollback. The same runbook runs in each environment; what changes is the go/no-go gate that must pass before the change moves up.

Patch vs upgrade — same discipline, different scope

A **patch** (single fix, rollup, service pack, or WebLogic PSU) is a targeted change applied with OPatch or the OUAF install utilities and is generally reversible with a rollback. A **version upgrade** (for example C2M to 25.4) is a larger, forward operation that moves the release and often the certified stack; it is reversed by restoring from backup rather than a simple rollback. This runbook applies the same discipline to both — readiness, backup, apply, validate, rollback/restore — and calls out where they differ.

What this is not

This runbook governs the operation of patching and upgrading. It does not invent patch numbers or reproduce entitlement-gated content: specific patch IDs, service-pack numbers, and step-by-step release instructions live in each patch's README and in the release's Installation/Upgrade guide on My Oracle Support, which this runbook points you to. The included scripts are read-only where they touch live

systems (discovery, validation, reporting); the one remediation helper runs dry-run by default and never applies a patch itself.

The deployment model

> *Assess readiness and back up, apply the patch or upgrade in the lowest environment, prove it, validate against the gate, then promote up to production. The repeatable way: Assess, then Configure, then Pilot, then Validate, then Expand — with a verified backup and a documented rollback at every step.*

2. Requirements Guide — Oracle C2M & WebLogic Patching & Upgrade Runbook (to 25.4)

Verify every requirement before you apply a patch or start an upgrade. Each entry: Description, Why, Verify, Implement-if-missing. Category-wide prerequisites are in `global-requirements-oracle.md`.

> *Version note: C2M 25.4 / minimum OUAF v2.9 / WebLogic 14c are the in-scope versions — **verify for your version** against the release's certification and upgrade documentation on My Oracle Support (MOS). Specific patch numbers and MOS notes are pointed to, not reproduced.*

2.1 My Oracle Support (MOS) access and the patch/upgrade media

Description: A valid Oracle support entitlement and My Oracle Support access to download the C2M/OUAF patches, WebLogic patches (OPatch patch ZIPs), and — for an upgrade — the target release installation/upgrade media and its README/upgrade guide. Why: Patches, media, and the certification matrix all flow from the entitlement; without MOS access you cannot obtain the fix or read the patch README that governs how it is applied. Verify: You can sign in to MOS, locate a patch by number for your platform, and download it; run `OC2MPATCH-Discovery.sh` to record the current patch/version baseline you will patch from. Implement-if-missing: Obtain the support entitlement and MOS credentials before proceeding; do not substitute an unofficial patch source.

2.2 Certified target release and stack

Description: A confirmed **target** C2M release (for a patch, your current release; for an upgrade, e.g. C2M 25.4) and an OS / JDK / Oracle Database / WebLogic combination **certified** for it per the release's certification matrix. Why: C2M/OUAF only supports certified combinations; patching or upgrading into an uncertified stack fails and loses supportability. Verify: Compare the environment's OS, JDK, DB, and WebLogic versions against the target release's certification matrix (MOS); run `OC2MPATCH-Discovery.sh` to capture the installed versions to compare. Implement-if-missing: Align OS/JDK/DB/WebLogic to a certified combination first — treat that alignment as its own change, backed up and validated, before the C2M patch/upgrade.

2.3 Verified backups (domain config, OUAF product/config, database)

Description: A **verified, restorable** backup taken immediately before any apply: the WebLogic domain configuration and Oracle Home, the OUAF product and configuration (splenviron/config), and the C2M database (or a guaranteed DB restore point/RMAN backup). Why: A patch is only safely reversible if you can get back to the pre-patch state; an upgrade is reversed by restore, not rollback. A backup you have not

verified is not a backup. Verify: The backup completed and a restore has been proven at least once in a lower environment; run `OC2MPATCH-Validation.sh --pre-check` to confirm backup markers are present before apply. Implement-if-missing: Establish and test the backup/restore path (DB restore point + RMAN, Oracle Home and domain archive, OUAF config archive) before touching any environment.

2.4 The patch README / upgrade guide read and understood

Description: The specific patch README (for OPatch or OUAF fixes) or the release upgrade guide has been read, including prerequisites, ordering (database component before app-server component for OUAF), conflicts, and the documented rollback. Why: Each patch/upgrade has its own prerequisites and order; applying without reading the README is the most common cause of a failed or half-applied change. Verify: You can state the patch's prerequisites, whether it conflicts with an applied patch (`opatch lsinventory` / OUAF inventory), and its documented rollback path. Implement-if-missing: Download and read the README/upgrade guide from MOS before scheduling the change.

2.5 Change control and a maintenance window

Description: An approved **change record** and a **maintenance window** for each environment, with production changes gated on successful lower-environment validation. Why: Patching and upgrading are high-impact; production changes must be approved, scheduled, and communicated, and must not proceed until lower environments have passed their gate. Verify: A change record exists with an approved window, a rollback plan, and named approvers; the lower environments' validation results are attached. Implement-if-missing: Raise the change with the rollback plan and window; do not apply to production without approval and a passing lower-environment gate.

2.6 A non-production pilot environment

Description: A representative **non-production** environment (Dev, then Test/QA/UAT) that mirrors production closely enough (same certified stack, comparable data) to prove the patch/upgrade first. Why: Every patch and upgrade is proven in a lower environment before production; the pilot is where you catch conflicts, data issues, and rollback problems cheaply. Verify: A pilot environment exists at the same certified stack as production; run `OC2MPATCH-ScopeHelper.sh` to confirm the environment order and produce the target list for this change. Implement-if-missing: Stand up or refresh a non-production environment to a production-like state before scheduling the production change.

2.7 Administrative access and tooling (OPatch, OUAF utilities, SQL, WLST)

Description: Administrative access and the toolchain: **OPatch** in the WebLogic Oracle Home, the OUAF utilities (`configureEnv.sh`, `initialSetup.sh`, `installSF.sh`/`installSFgroup.sh`, `ouafDatabasePatch.sh`, `spl.sh`), **WLST (Jython)** for WebLogic, and **SQL*Plus / SQLcl** for the database. Why: The runbook applies and validates changes through these interfaces; without them you cannot apply, inventory, or verify a patch. Verify: `opatch version` runs from the Oracle Home, the OUAF product account can run the framework utilities, WLST connects to the domain, and SQL connects to the C2M schema; run `OC2MPATCH-Discovery.sh` (exercises the read-only paths of each). Implement-if-missing: Provision the access and install/locate the client tooling before the change.

2.8 A go/no-go gate and rollback plan per environment

Description: A defined pass/fail **gate** (what must be true to promote the change up) and a documented **rollback** (OPatch rollback / OUAF back-out for patches; restore-from-backup for upgrades) for each environment. Why: Promotion between environments must be a decision against a gate, not a habit; and every apply must have a rehearsed way back. Verify: The gate criteria and rollback steps are written into the change record; run `OC2MPATCH-Validation.sh` in a lower environment to confirm the gate can be evaluated. Implement-if-missing: Author the gate from the Testing Guide and the rollback from the patch README before the first apply.

Checklist

- MOS access + patch/upgrade media obtained — `OC2MPATCH-Discovery.sh`
- Certified target release and stack confirmed
- Verified, restorable backups taken — `OC2MPATCH-Validation.sh --pre-check`
- Patch README / upgrade guide read (prereqs, order, rollback)
- Change record + maintenance window approved
- Non-production pilot environment ready — `OC2MPATCH-ScopeHelper.sh`
- Admin access + tooling verified (OPatch, OUAF utilities, WLST, SQL) — `OC2MPATCH-Discovery.sh`
- Go/no-go gate + rollback plan documented per environment — `OC2MPATCH-Validation.sh`

Global Requirements — Oracle Utilities C2M / WebLogic Solutions

These requirements apply to **every** solution in the **Oracle** category (Oracle Utilities Customer to Meter and its Oracle WebLogic Server middleware). Verify each before starting a solution's steps.

> ***Vendor & version boundary (applies to every Oracle solution).*** *Oracle Utilities* ***Customer to Meter (C2M)****,* ***Customer Care & Billing (CC&B)****,* ***Meter Data Management (MDM)****, and* ***Oracle WebLogic Server*** *are authored from Oracle's official product documentation and the Oracle Utilities Application Framework (OUAF) install/admin guides.* ***Version-specific details*** *(C2M 25.4 / minimum v2.9, WebLogic 14c) are* ***cited or flagged "verify for your version"*** *— never invented. Some Oracle documentation and patches are entitlement-gated on* ***My Oracle Support (MOS)****; where a step requires a MOS note, patch number, or download, the solution points to it rather than reproducing gated content. All work assumes a* ***non-production environment first*** *and change control for production.*

1. Licensed Oracle Utilities C2M + WebLogic entitlement

- **Description:** A licensed Oracle Utilities **Customer to Meter** (and the underlying OUAF) plus the supported **Oracle WebLogic Server 14c** and database (Oracle Database) at the versions in scope.
- **Why required:** Installation media, patches, and support all flow from the entitlement.
- **How to verify:** Confirm the license and access to **My Oracle Support**; check the certified

version matrix for C2M ↔ WebLogic ↔ Database ↔ Java.

- **How to implement if missing:** Obtain licensing/entitlement and MOS access before proceeding.

2. Certified platform stack (OS, Java, Database, WebLogic)

- **Description:** A host OS, JDK, Oracle Database, and WebLogic version combination **certified** for the target C2M release (per Oracle's certification/compatibility matrix).
- **Why required:** C2M/OUAF only supports certified combinations; uncertified stacks cause install and patch failures and lose supportability.
- **How to verify:** Compare the environment against the C2M release's certification matrix (MOS).
- **How to implement if missing:** Align OS/JDK/DB/WebLogic to a certified combination first.

3. Environment topology (Dev / Test / Prod) & access

- **Description:** Defined **Development, Test, and Production** environments (and any QA/UAT), with OS accounts (e.g. the OUAF/`splunk`-style product user), WebLogic admin credentials, and DB schema access appropriate to the task.
- **Why required:** C2M is deployed and promoted across environments; each solution assumes the target environment exists and you can reach WebLogic, the file system, and the database.
- **How to verify:** Confirm you can log in to the OS product account, the WebLogic Administration Console / WLST, and the C2M database schema.
- **How to implement if missing:** Stand up / obtain access to the environment before starting.

4. Administrative access & tooling (WLST, shell, SQL)

- **Description:** Administrative access and the standard toolchain: **WLST (Jython)** for WebLogic, the OS **shell** (bash/ksh) for OUAF utilities (e.g. `splmanager`, `configureEnv`, `initialSetup`, batch/threadpool scripts), and **SQL*Plus / SQLcl** for the database.
- **Why required:** C2M/WebLogic administration is performed through these interfaces, not a GUI-only workflow; the solutions' automation is authored as WLST/shell/SQL.
- **How to verify:** Confirm WLST connects to the domain, shell access to the product home, and SQL connectivity to the schema.
- **How to implement if missing:** Provision the access and install the client tooling.

5. Change control, backup & a non-production pilot

- **Description:** A **change-control** process for production changes, verified **backups** (domain

config, OUAF product/config, database) before disruptive actions (install, patch, upgrade, migration, tuning), and a **non-production environment** to validate on first.

• **Why required:** Every solution follows **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** with a documented rollback; C2M patches/upgrades and WebLogic changes are high-impact.

• **How to verify:** Confirm a backup/restore path and a change window/approval for production.

• **How to implement if missing:** Establish backups and change control before touching production.

6. Collaborating teams & support model

• **Description:** Engagement with the **infrastructure, DBA, network, application-development, and release/test **teams, and (post go-live) the** hypercare / on-call** support model and monitoring stack (e.g. Dynatrace) the environment uses.

• **Why required:** C2M administration is cross-functional; issues span middleware, database, network, and application layers, and production support runs on a defined escalation and monitoring model.

• **How to verify:** Confirm the named contacts, escalation path, and monitoring tooling.

• **How to implement if missing:** Identify the teams and the support/monitoring model before go-live.

*The deployment model across every Oracle solution is the same: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand**, with explicit pass/fail gates and a documented rollback for production changes. Automation is delivered as **WLST (Jython), shell, and SQL** scripts — not PowerShell.*

3. Upgrade & Patch Runbook — Oracle C2M & WebLogic Patching & Upgrade Runbook (to 25.4)

Follow these steps in order, **per environment**, lowest first (Dev → Test/QA/UAT → Production). The runbook is the same in each environment; what changes is the go/no-go gate that must pass before the change is promoted up. Nothing is applied to production until every lower environment has passed its gate. Exact patch numbers, service-pack IDs, and release upgrade instructions live in the patch README and the release upgrade guide on My Oracle Support (MOS) — read them first; this runbook is the method, not a substitute for the vendor's per-patch instructions.

> *Model: prove the change in the lowest environment, back up before every apply, validate against a gate, then promote up. The repeatable way: Assess, then Configure, then Pilot, then Validate, then Expand — with a verified backup and a documented rollback at every step. Placeholders below use plain names (your-oracle-home, your-env) — substitute your environment's values. Commands are illustrative of the OPatch / OUAF / WLST / SQL interfaces; always follow the specific patch README.*

Phase A — Readiness and backup (every environment, before any apply)

Step 1 — Establish the current baseline (Assess)

1. Run `OC2MPATCH-Discovery.sh` to capture, read-only, the current state: C2M/OUAF release and applied

fixes (OUAF inventory), WebLogic Oracle Home patches (`opatch lsinventory`), OS/JDK/DB/WebLogic versions, and domain/managed-server layout. Keep the output — it is the baseline you patch from and compare against afterward.

1. Confirm the change target: for a patch, the patch number(s) and their README; for an upgrade, the

target release (e.g. C2M 25.4) and its upgrade guide.

Step 2 — Confirm certification and prerequisites

1. Compare the discovered OS/JDK/DB/WebLogic versions against the target release's certification matrix

(MOS). If the stack is not certified for the target, align it first (its own backed-up change).

1. From the patch README / upgrade guide, confirm prerequisites and conflicts:

`opatch lsinventory` for WebLogic conflicts; the OUAF fix inventory for C2M conflicts.

1. Confirm the environment order and produce the target list for this change with

`OC2MPATCH-ScopeHelper.sh`.

Step 3 — Take verified backups

1. Stop the environment cleanly for a consistent backup where required: `spl.sh -stop` (OUAF) and stop

the WebLogic managed/admin servers (or use the runbook's WLST stop). OUAF shuts WebLogic down automatically during install, but back up from a known-stopped state.

1. Back up, and verify each is restorable:

• Database: create a guaranteed restore point and/or RMAN backup of the C2M schema/database.

• WebLogic: archive the Oracle Home and the domain configuration.

• OUAF: archive the product home and configuration (splenviron/config).

1. Run `OC2MPATCH-Validation.sh --pre-check` to confirm the backup markers are present. Do not proceed

past a missing or unverified backup.

> ***Gate A (readiness):*** *baseline captured, target certified, prerequisites/conflicts cleared, and verified backups present. Do not apply until Gate A passes.*

Phase B — Apply the patch / perform the upgrade

Step 4 — Apply WebLogic (OPatch) patches

1. Set the environment to the WebLogic Oracle Home (your-oracle-home) and unzip the patch to a staging

directory outside the Oracle Home.

1. From the patch directory, verify then apply per the README:

• `opatch lsinventory` — record the pre-apply inventory.

• `opatch apply` — apply the patch (run from the unzipped patch directory; follow README ordering for multiple patches).

• `opatch lsinventory` — confirm the patch now shows as applied.

1. If OPatch reports a conflict, stop and resolve per the README (do not force) — this is a no-go for this environment until resolved.

Step 5 — Apply C2M / OUAF patches

1. As the OUAF product account, source the environment (`configureEnv.sh` / splenviron) for your-env.

2. Apply per the README, database component first then application-server component:

• Single/group software fixes: `installSF.sh` / `installSFgroup.sh`.

• Database patches: `ouafDatabasePatch.sh`.

• Apply configuration: `initialSetup.sh`.

1. Let the OUAF error-checker run at the end of install; a failed error-check is a no-go for this environment.

Step 6 — Perform a version upgrade (when the change is an upgrade, not a patch)

1. Follow the target release's **upgrade guide** (e.g. C2M 25.4): install the new OUAF/C2M version against the environment, run the upgrade/`initialSetup.sh` steps, and apply any post-upgrade fixes the guide specifies.

1. An upgrade moves the release (and often the certified stack). Its "rollback" is **restore from the Phase A backup**, not an OPatch/OUAF back-out — confirm the restore path before you start.

1. Redeploy/restart the WebLogic domain and C2M as the guide directs.

Step 7 — Restart and bring the environment up

1. Start WebLogic (admin then managed servers) and the OUAF environment: `spl.sh -start`.

2. Confirm all managed servers reach RUNNING (WLST or the Administration Console) before validation.

Phase C — Post-apply validation (every environment)

Step 8 — Validate the applied change

1. Run `OC2MPATCH-Validation.sh` to confirm, read-only: the expected patch/version now shows in the OUAF and OPatch inventories, managed servers are RUNNING, key C2M health checks pass, and the database is at the expected level.

1. Run `OC2MPATCH-Report.sh` to produce the patch/version coverage report for this environment (baseline vs post-apply, per node), for the change record.

1. Execute the functional smoke tests in the Testing Guide for this environment.

> ***Gate C (post-apply):*** *inventories show the expected change, servers RUNNING, health and smoke tests pass, report clean. Only when Gate C passes is the change promoted to the next environment up.*

Phase D — Rollback (if a gate fails)

Step 9 — Roll back or restore

1. For a **patch** that failed validation: `opatch rollback` (WebLogic) and/or the OUAF back-out per the README, then re-run `OC2MPATCH-Validation.sh` to confirm the environment is back at baseline.

1. For an **upgrade** that failed validation, or a patch with no clean back-out: **restore from the Phase A backup** (DB restore point/RMAN, Oracle Home, domain, OUAF config), restart, and re-validate against the baseline.

1. Record the failure, the root cause, and the rollback outcome in the change record. Do not promote a change that failed a lower-environment gate.

Phase E — Promote up, then production

Step 10 — Promote through the environment progression

1. After a lower environment passes Gate C, repeat Phases A–C in the next environment up (Test/QA/UAT), inside its change window, with its own fresh backup.

1. Apply to **Production** only after every lower environment has passed its gate — inside the approved production maintenance window, with a fresh verified backup and the rehearsed rollback ready.

1. After production passes Gate C, run `OC2MPATCH-Report.sh` for the final coverage record and close the change.

Order of operations

```
Baseline -> Certify + prereqs -> Backup (Gate A) -> Apply WebLogic/OUAF or
Upgrade -> Restart -> Validate (Gate C) -> [Rollback if fail] -> Promote up ->
Production
```

5. Testing Guide — Oracle C2M & WebLogic Patching & Upgrade Runbook (to 25.4)

Run in each environment after the apply (Runbook Phase C) and before promoting the change up. Each test confirms one part of the patch/upgrade landed correctly and the environment is healthy. The pass/fail gate at the end is the go/no-go for promoting to the next environment.

Test 1 — Backup was verified before apply (readiness)

Do: Confirm the Phase A backups exist and a restore was proven; run `OC2MPATCH-Validation.sh --pre-check`. Expected: Database restore point/RMAN, Oracle Home, domain, and OUAF config backup markers are present; the pre-check reports PASS. (This test gates the apply, not the promotion — it must have passed before Phase B.)

Test 2 — WebLogic patch inventory shows the change

Do: Run `opatch lsinventory` from the WebLogic Oracle Home (or read it via `OC2MPATCH-Validation.sh`). Expected: The expected patch number(s) appear as applied for a patch, or the expected release/OPatch level for an upgrade; no unexpected conflicts.

Test 3 — C2M / OUAF inventory shows the change

Do: Run `OC2MPATCH-Validation.sh` and check the OUAF fix/version inventory. Expected: The expected C2M/OUAF fix or target release (e.g. 25.4) shows as installed; the OUAF error-checker reported clean at install.

Test 4 — Managed servers reach RUNNING

Do: After `spl.sh -start` and the WebLogic start, check server state via WLST or the Administration Console (or `OC2MPATCH-Validation.sh`). Expected: The admin server and all managed servers are RUNNING; no server stuck in ADMIN/FAILED.

Test 5 — Database is at the expected level

Do: Run the database checks in `OC2MPATCH-Validation.sh` (release/patch-level query against the C2M schema). Expected: The database component reports the expected post-patch/upgrade level; no invalid objects introduced by the change.

Test 6 — C2M functional smoke test

Do: Log in to C2M and exercise a representative transaction path for the environment (a read and a core business function per your smoke-test set). Expected: The application responds normally; no new errors in the logs attributable to the change.

Test 7 — Coverage report is clean and matches baseline intent

Do: Run `OC2MPATCH-Report.sh` and compare against the Phase A baseline from `OC2MPATCH-Discovery.sh`. Expected: The report shows the intended patch/version delta per node and no unexpected drift between nodes in the environment.

Test 8 — Rollback path is confirmed available

Do: Confirm (do not execute unless a gate fails) the documented rollback for this change: `opatch rollback` / OUAF back-out for a patch, or restore-from-backup for an upgrade. Expected: The rollback steps are written into the change record and the backup needed for restore is present and verified.

Pass/fail gate

Gate	Requirement
Apply gate (Gate A)	Test 1 passes — verified backups present — before any Phase B apply
Promote gate (Gate C)	Tests 2–7 pass in this environment and Test 8 (rollback available) is confirmed, before promoting the change to the next environment up
Production gate	Every lower environment passed its promote gate; Tests 2–7 pass in Production inside the approved window

6. Troubleshooting Guide — Oracle C2M & WebLogic Patching & Upgrade Runbook (to 25.4)

Use `OC2MPATCH-Discovery.sh` for the current baseline, `OC2MPATCH-Validation.sh` for post-apply and pre-check state, `OC2MPATCH-Report.sh` for the coverage view, and `OC2MPATCH-Remediation.sh` (dry-run by default) for safe fixes to common gaps. Always read the specific patch README first — patch-specific errors are documented there. Specific patch numbers and MOS notes are pointed to, not reproduced.

A. OPatch apply fails or reports a conflict

Symptom	Cause	Remediation
`opatch apply` reports a conflict with an applied patch	The new patch overlaps an already-applied WebLogic patch	Stop; run `opatch lsinventory`; resolve per the patch README (superseding/merge patch from MOS). Do not force — a forced apply leaves an unsupported Oracle Home
`opatch` cannot find the Oracle Home / inventory	Central Inventory missing or `ORACLE_HOME` not set	Set the environment to your-oracle-home; confirm the inventory with `opatch lsinventory -detail`; repair the Central Inventory per Oracle docs before retrying
Wrong OPatch version	OPatch older than the patch requires	Update OPatch in the Oracle Home to the version the README/release requires (verify for your version), then retry

B. OUAF patch install fails

Symptom	Cause	Remediation
Install stops at the database component	DB patch prerequisite or connectivity	Confirm SQL connectivity to the C2M schema and the DB patch prerequisite; apply the database component (`ouafDatabasePatch.sh`) before the app-server component
Error-checker reports failures at end of install	A component did not apply cleanly	Review the install log; re-apply per the README; a failed error-check is a no-go — do not promote the environment
Environment did not shut down for install	Non-WebLogic env, or manual stop needed	OUAF stops WebLogic automatically; for non-WebLogic environments stop manually with `spl.sh -stop` before install

C. Managed servers do not reach RUNNING after apply

Symptom	Cause	Remediation
Server stuck in ADMIN or FAILED	Config/patch mismatch or startup error	Check the server log; confirm the domain config matches the applied patch; if attributable to the change, roll back (Phase D) and re-validate against baseline
Admin server starts, managed servers do not	Node manager / config drift after patch	Verify node manager and the managed-server config via WLST; restart in order (admin, then managed)

D. Backup/pre-check gate will not pass

- `OC2MPATCH-Validation.sh --pre-check` reports a missing backup marker. Do **not** apply. Take the

missing backup (DB restore point/RMAN, Oracle Home, domain, OUAF config), verify it is restorable, and re-run the pre-check. A missing or unverified backup is an absolute no-go for the apply.

E. Stack is not certified for the target release

- `OC2MPATCH-Discovery.sh` shows an OS/JDK/DB/WebLogic version outside the target release's

certification matrix. Align the stack to a certified combination first — as its own backed-up, validated change — before applying the C2M patch/upgrade. Patching into an uncertified stack loses supportability.

F. Version/patch drift between nodes

• `OC2MPATCH-Report.sh` shows different patch/version levels across nodes in one environment. Re-apply

the missing patch to the lagging node per the README; never leave an environment with mixed levels — it is unsupported and produces intermittent, hard-to-diagnose behavior.

G. A change failed validation — rollback

• For a **patch**: `opatch rollback` (WebLogic) and/or the OUAF back-out per the README, then re-run

`OC2MPATCH-Validation.sh` to confirm the environment is back at baseline.

• For an **upgrade** (or a patch with no clean back-out): restore from the Phase A backup (DB restore

point/RMAN, Oracle Home, domain, OUAF config), restart, and re-validate against baseline. Record the root cause; do not promote a change that failed a lower-environment gate.

H. Diagnostics quick reference

```
./OC2MPATCH-Discovery.sh                # read-only baseline: OUAF fixes, opatch lsinventory, versions
./OC2MPATCH-Validation.sh --pre-check   # confirm verified backups before apply (Gate A)
./OC2MPATCH-Validation.sh               # post-apply: inventory, servers RUNNING, DB level (Gate C)
./OC2MPATCH-Report.sh                   # patch/version coverage per node (baseline vs post-apply)
./OC2MPATCH-Remediation.sh              # dry-run by default; add --apply to act on approved fixes
./OC2MPATCH-ScopeHelper.sh              # environment/target list + lower-first progression
```

I. When to escalate

If backups are verified, the stack is certified, the README prerequisites are met, and the patch still fails to apply or validate after a clean rollback, capture the `opatch lsinventory` output, the OUAF install log, and the `OC2MPATCH-Validation.sh` / `OC2MPATCH-Report.sh` output, and open a service request on My Oracle Support against the patch, quoting the patch README and the failing step.

7. Best Practices — Oracle C2M & WebLogic Patching & Upgrade Runbook (to 25.4)

• Lower environments first, always. Apply and validate in Dev, then Test/QA/UAT, and only reach

Production after every lower environment has passed its gate — production is never the first place a patch or upgrade runs.

• Back up before you touch anything, and verify the backup. A backup you have not proven restorable is

not a backup. Take the DB restore point/RMAN, Oracle Home, domain, and OUAF config backup, and gate the apply on `OC2MPATCH-Validation.sh --pre-check`.

• Read the patch README first, every time. Prerequisites, conflicts, ordering, and the documented

rollback are patch-specific; most failed or half-applied changes come from skipping the README.

• Stay certified. Confirm OS/JDK/DB/WebLogic is a certified combination for the target C2M release

before you patch or upgrade; align an uncertified stack as its own backed-up change first.

• Apply OUAF components in the right order. Database component before application-server component, then

`initialSetup.sh`, then let the error-checker run — a failed error-check is a no-go, not a warning.

• Never force past an OPatch conflict. If `opatch apply` reports a conflict, stop and resolve per the README; forcing leaves the Oracle Home in an unsupported state.

• Treat an upgrade's rollback as restore-from-backup. A version upgrade (e.g. to 25.4) is reversed by restoring the Phase A backup, not by a simple back-out — confirm the restore path before you start.

• Validate against the baseline, not from memory. Capture the pre-change baseline with `OC2MPATCH-Discovery.sh` and compare with `OC2MPATCH-Report.sh` afterward, per node, so drift between nodes is caught.

• Gate every promotion on a decision. Promotion between environments is a go/no-go against a written gate, never a habit; a change that fails a lower gate does not move up.

• Do not invent patch numbers or apply unofficial fixes. Obtain patches and their READMEs from My Oracle Support only; point to the MOS note rather than reproducing gated content.

• Change control and a window for production. Every production apply runs inside an approved window, with a fresh backup and the rehearsed rollback ready, communicated to the support and business teams.

• Keep read-only tools read-only. Use `OC2MPATCH-Discovery.sh`, `OC2MPATCH-Validation.sh`, and

`OC2MPATCH-Report.sh` to observe; keep `OC2MPATCH-Remediation.sh` in dry-run until a fix is deliberately approved.

8. Deliverables Summary — Oracle C2M & WebLogic Patching & Upgrade Runbook (to 25.4)

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box
1. Solution Overview	The C2M/OUAF + WebLogic patch/upgrade model, supportability, the certified-version matrix, OPatch/OUAF tooling, and the environment progression (the why)
2. Requirements Guide	Every prerequisite with Verify / Implement-if-missing
Global Requirements (Oracle)	Category-wide prerequisites, published once
3. Upgrade & Patch Runbook	Exact, ordered steps: readiness + backup, apply patches / perform upgrade, post-apply validation, rollback — per environment with go/no-go gates

Document	Purpose
5. Testing Guide	Validation procedures with pass/fail gates
6. Troubleshooting Guide	Field issues and remediation
7. Best Practices	Field-proven recommendations

Automation scripts (WLST/Jython, shell & SQL — not PowerShell)

Script	What it does
`OC2MPATCH-Discovery.sh`	Read-only baseline of current patch/version state: C2M/OUAF fix inventory, WebLogic `opatch lsinventory`, OS/JDK/DB/WebLogic versions, domain/server layout
`OC2MPATCH-Validation.sh`	Read-only post-patch validation: expected patch/version present, managed servers RUNNING, DB level, health checks; `--pre-check` confirms backups before apply
`OC2MPATCH-Report.sh`	Read-only patch/version coverage report per environment/node (baseline vs post-apply) for the change record
`OC2MPATCH-Remediation.sh`	Safe remediation helper for common gaps; **dry-run by default**, never applies a patch itself
`OC2MPATCH-ScopeHelper.sh`	Builds the environment/target list and confirms the lower-first progression for this change

Named runbook artifacts

Artifact	Name / description
Readiness gate	`Gate A` — verified backups + certified stack + cleared prerequisites before any apply
Promote gate	`Gate C` — inventories/health/tests pass before promoting the change to the next environment up
Backup set	DB restore point/RMAN + Oracle Home + WebLogic domain + OUAF product/config archives
Rollback	OPatch rollback / OUAF back-out (patch) or restore-from-backup (upgrade)

What the buyer receives

- This runbook as a branded Word document and PDF.
- All five automation scripts (shell / WLST-Jython / SQL) in a single ZIP.

- A repeatable patch/upgrade method following the model: Assess, then Configure, then Pilot, then Validate, then Expand — proven in lower environments before production.

Outcome

C2M and WebLogic kept current and supportable — patches and version upgrades (toward 25.4) applied with readiness, backup, validation, and a documented rollback — proven in lower environments before production, closing support gaps without unplanned outages.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

OC2MPATCH-Discovery.sh

Purpose: Capture, read-only, the state you patch from and compare against afterward: the C2M/OUAF release and applied fixes, the WebLogic Oracle Home patch inventory (opatch lsinventory), the OS/JDK/DB/WebLogic versions, and the domain/server layout. Set the environment for your-oracle-home and your-env before running. Reads live systems only; changes nothing. HOW TO RUN ./OC2MPATCH-Discovery.sh Run as the OUAF product account with OPatch and the OUAF utilities on PATH. EXPECTED RESULTS A baseline summary printed to stdout: OUAF release + fix list, opatch lsinventory patches, OS/JDK/DB/WebLogic versions, and managed-server list. Each source prints PASS (readable) or GAP (not reachable — record against the Requirements checklist). Save the output as the baseline. NOTES Oracle — C2M/WebLogic patching. Read-only against live systems. The repeatable way: this is the same baseline capture run in every environment before any apply.

How to run: ./OC2MPATCH-Discovery.sh Run as the OUAF product account with OPatch and the OUAF utilities on PATH. EXPECTED RESULTS A baseline summary printed to stdout: OUAF release + fix list, opatch lsinventory patches, OS/JDK/DB/WebLogic versions, and managed-server list. Each source prints PASS (readable) or GAP (not reachable — record against the Requirements checklist). Save the output as the baseline. NOTES Oracle — C2M/WebLogic patching. Read-only against live systems. The repeatable way: this is the same baseline capture run in every environment before any apply.

OC2MPATCH-Remediation.sh

Purpose: Address the small, safe gaps a validation run surfaces — a stopped environment that should be running, a managed server not started, missing backup directories before an apply, or an OUAF configuration not yet applied. It DRY-RUNS by default: it prints the exact action it would take and takes none. It NEVER applies a patch or performs an upgrade — patches/upgrades are applied by the runbook with OPatch/OUAF utilities after the README, not by this helper. Set the environment for your-oracle-home and your-env before running. HOW TO RUN ./OC2MPATCH-Remediation.sh (dry-run — prints intended actions only) ./OC2MPATCH-Remediation.sh --apply (act on the approved fixes) Run as the OUAF product account. Only use --apply after the fix is deliberately approved. EXPECTED RESULTS Dry-run: a numbered list of intended actions, each prefixed [DRY-RUN], with no change made. --apply: each action runs and prints PASS/FAIL. No patch is ever applied by this script. NOTES Oracle — C2M/WebLogic patching. Dry-run by default; --apply gates real actions. It never applies a patch/upgrade — that stays a deliberate runbook step. The repeatable way.

How to run: ./OC2MPATCH-Remediation.sh (dry-run — prints intended actions only) ./OC2MPATCH-Remediation.sh --apply (act on the approved fixes) Run as the OUAF product account. Only use --apply after the fix is deliberately approved. EXPECTED RESULTS Dry-run: a numbered list of intended actions, each prefixed [DRY-RUN], with no change made. --apply: each action runs and prints PASS/FAIL. No patch is ever applied by this script. NOTES Oracle — C2M/WebLogic patching. Dry-run by default; --apply gates real actions. It never applies a patch/upgrade — that stays a deliberate runbook step. The repeatable way.

OC2MPATCH-Report.sql

Purpose: Produce the database side of the patch/version coverage record for the change: the Oracle Database version, the OUAF/C2M schema release and applied-fix rows, and a count of invalid objects (a post-patch health signal). Run against the C2M schema. Read-only SELECTs only; it makes no changes. Pair its output with the WebLogic/OS baseline from OC2MPATCH-Discovery.sh. HOW TO RUN sqlplus your-c2m-schema/your-password@your-tns @OC2MPATCH-Report.sql (or: sql your-c2m-schema/your-password@your-tns @OC2MPATCH-Report.sql) EXPECTED RESULTS A printed report: DB version, C2M/OUAF schema release/fixes, and invalid-object count. Compare release/fixes to the intended target (e.g. 25.4). Invalid-object count should not have grown because of the change; investigate any increase before promoting (Gate C). NOTES Oracle — C2M/WebLogic patching. Read-only SQL. The repeatable way: run the same report in every environment, baseline vs post-apply, so drift between nodes/environments is caught. Table/view names for the OUAF fix inventory vary by release — verify for your version.

How to run: sqlplus your-c2m-schema/your-password@your-tns @OC2MPATCH-Report.sql (or: sql your-c2m-schema/your-password@your-tns @OC2MPATCH-Report.sql) EXPECTED RESULTS A printed report: DB version, C2M/OUAF schema release/fixes, and invalid-object count. Compare release/fixes to the intended target (e.g. 25.4). Invalid-object count should not have grown because of the change; investigate any increase before promoting (Gate C). NOTES Oracle — C2M/WebLogic patching. Read-only SQL. The repeatable way: run the same report in every environment, baseline vs post-apply, so drift between nodes/environments is caught. Table/view names for the OUAF fix inventory vary by release — verify for your version.

OC2MPATCH-ScopeHelper.py

Purpose: Build the ordered environment/target list for a patch or upgrade and confirm the lower-first progression (Dev -> Test/QA/UAT -> Production), and — when connected to a domain — read the admin/managed-server layout and state so the change targets the right nodes. Read-only WLST: it connects, reads server state, and prints the plan; it changes no configuration. Set the connection values for your-admin-url and your-env before running. HOW TO RUN wlst.sh OC2MPATCH-ScopeHelper.py Provide connection details via environment variables (see below) or edit the defaults. Read-only: it does not edit, deploy, or apply anything. EXPECTED RESULTS A printed plan: the environment order with Production last, and — if a connection is made — the domain's admin and managed servers with their current state (RUNNING/ADMIN/etc.), so you confirm the correct, healthy target list before Phase B. NOTES Oracle — C2M/WebLogic patching. Read-only WLST (Jython). The repeatable way: use the same ordered progression for every change so production is never patched before lower environments.

How to run: wlst.sh OC2MPATCH-ScopeHelper.py Provide connection details via environment variables (see below) or edit the defaults. Read-only: it does not edit, deploy, or apply anything. EXPECTED RESULTS A printed plan: the environment order with Production last, and — if a connection is made — the domain's admin and managed servers with their current state (RUNNING/ADMIN/etc.), so you confirm the correct, healthy target list before Phase B. NOTES Oracle — C2M/WebLogic patching. Read-only WLST (Jython). The repeatable way: use the same ordered progression for every change so production is never patched before lower environments.

OC2MPATCH-Validation.sh

Purpose: Two read-only modes. With --pre-check it confirms the Phase A backup markers exist before an apply (Gate A). Without it, it validates after the apply (Gate C): the expected patch/version shows in the OPatch and OUAF inventories, WebLogic managed servers are RUNNING, and the database reports the

expected level. Set the environment for your-oracle-home and your-env. Reads live systems only; changes nothing. HOW TO RUN ./OC2MPATCH-Validation.sh --pre-check (before apply — confirm verified backups) ./OC2MPATCH-Validation.sh (after apply — confirm the change landed) Run as the OUAF product account; needs SQL access to the C2M schema and WLST for server state. EXPECTED RESULTS Pre-check: each backup marker prints PASS/GAP; any GAP is an absolute no-go for the apply. Post-apply: each check prints PASS/FAIL; all PASS is the Gate C go decision to promote up. NOTES Oracle — C2M/WebLogic patching. Read-only against live systems. The repeatable way: run the pre-check before every apply and the post-apply validation before every promotion.

How to run: ./OC2MPATCH-Validation.sh --pre-check (before apply — confirm verified backups) ./OC2MPATCH-Validation.sh (after apply — confirm the change landed) Run as the OUAF product account; needs SQL access to the C2M schema and WLST for server state. EXPECTED RESULTS Pre-check: each backup marker prints PASS/GAP; any GAP is an absolute no-go for the apply. Post-apply: each check prints PASS/FAIL; all PASS is the Gate C go decision to promote up. NOTES Oracle — C2M/WebLogic patching. Read-only against live systems. The repeatable way: run the pre-check before every apply and the post-apply validation before every promotion.